

Лабораторная работа № 2

Первоначальная настройка git

Пириев Керем

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
3.1	1. Установка программного обеспечения	7
3.2	2. Базовая настройка git	8
3.3	3. Создание SSH ключей	8
3.4	4. Создание PGP ключа (GPG)	9
3.5	5. Добавление SSH ключа на GitHub	11
3.6	6. Добавление GPG ключа на GitHub	11
3.7	7. Настройка подписи коммитов git	11
3.8	8. Создание репозитория на GitHub	12
3.9	9. Клонирование репозитория	12
3.10	10. Настройка каталога курса	13
4	Вывод	14
5	Контрольные вопросы	15
5.1	1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются?	15
5.2	2. Объясните следующие понятия VCS и их отношения: хранилище, commit, история, рабочая копия.	15
5.3	3. Что представляют собой и чем отличаются централизованные и децентрализованные VCS? Приведите примеры VCS каждого вида.	16
5.4	4. Опишите действия с VCS при единоличной работе с хранилищем.	16
5.5	5. Опишите порядок работы с общим хранилищем VCS.	16
5.6	6. Каковы основные задачи, решаемые инструментальным средством git?	16
5.7	7. Назовите и дайте краткую характеристику командам git.	17
5.8	8. Приведите примеры использования при работе с локальным и удалённым репозиториями.	17
5.9	9. Что такое и зачем могут быть нужны ветви (branches)?	18
5.10	10. Как и зачем можно игнорировать некоторые файлы при commit?	18
	Список литературы	19

Список иллюстраций

3.1	Установка Git	7
3.2	Установка GitHub CLI	7
3.3	Базовая настройка git	8
3.4	Создание RSA ключа	8
3.5	Создание ключа ed25519	9
3.6	Просмотр публичного ключа SSH	9
3.7	Создание PGP ключа	10
3.8	Просмотр GPG ключей	10
3.9	Копирование и создание SSH ключа	11
3.10	Добавление GPG ключа и проверка ssh	11
3.11	Настройка подписи коммитов	12
3.12	Созданный репозиторий на GitHub	12
3.13	Клонирование репозитория	13
3.14	Отправка коммита	13

Список таблиц

1 Цель работы

Изучить идеологию и применение средств контроля версий, освоить умения по работе с git.

2 Задание

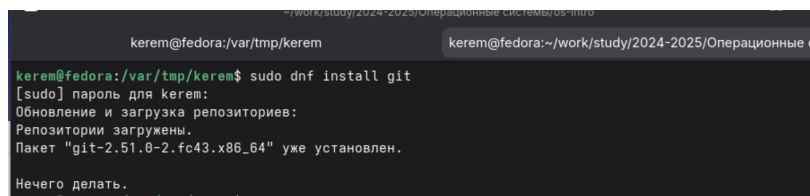
1. Установка программного обеспечения.
2. Базовая настройка git.
3. Создание SSH ключей.
4. Создание PGP ключа (GPG).
5. Добавление SSH ключа на GitHub.
6. Добавление GPG ключа на GitHub.
7. Настройка подписи коммитов git.
8. Создание репозитория на GitHub.
9. Клонирование репозитория.
10. Настройка каталога курса.

3 Выполнение лабораторной работы

3.1 1. Установка программного обеспечения

3.1.1 1.1 Установка Git

Git был установлен командой `sudo dnf install git` (рис. Рисунок 3.1).

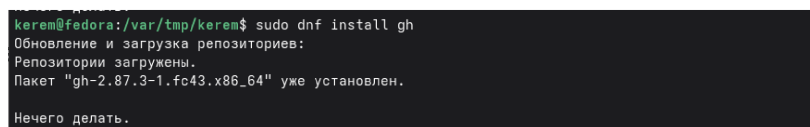


```
kerem@fedora:~/work/study/2024-2025/Операционные системы$  
kerem@fedora:/var/tmp/kerem$ sudo dnf install git  
[sudo] пароль для kerem:  
Обновление и загрузка репозитория:  
Репозитории загружены.  
Пакет "git-2.51.0-2.fc43.x86_64" уже установлен.  
Нечего делать.
```

Рисунок 3.1: Установка Git

3.1.2 1.2 Установка GitHub CLI (gh)

Был установлен пакет GitHub CLI командой `sudo dnf install gh` (рис. Рисунок 3.2).



```
kerem@fedora:/var/tmp/kerem$ sudo dnf install gh  
Обновление и загрузка репозитория:  
Репозитории загружены.  
Пакет "gh-2.87.3-1.fc43.x86_64" уже установлен.  
Нечего делать.
```

Рисунок 3.2: Установка GitHub CLI

3.2 2. Базовая настройка git

Были выполнены команды для глобальной настройки git (имя, email, ветка по умолчанию и параметры crlf) (рис. Рисунок 3.3).

```
kerem@fedora:/var/tmp/kerem$ git config --global user.name "kerem piriyew"
kerem@fedora:/var/tmp/kerem$ git config --global user.email "piriyewkerem144@gmail.com"
kerem@fedora:/var/tmp/kerem$ git config --global core.quotepath false
kerem@fedora:/var/tmp/kerem$ git config --global init.defaultBranch master
kerem@fedora:/var/tmp/kerem$ git config --global core.autocrlf input
kerem@fedora:/var/tmp/kerem$ git config --global core.safecrlf warn
```

Рисунок 3.3: Базовая настройка git

3.3 3. Создание SSH ключей

3.3.1 3.1 Создание ключей RSA и ed25519

Сначала был сгенерирован ключ RSA командой `ssh-keygen -t rsa -b 4096` (рис. Рисунок 3.4).

```
kerem@fedora:/var/tmp/kerem$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/kerem/.ssh/id_rsa):
/home/kerem/.ssh/id_rsa already exists.
Overwrite (y/n)? y
Enter passphrase for "/home/kerem/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kerem/.ssh/id_rsa
Your public key has been saved in /home/kerem/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:qRibr2gj6BYub7ebyEre/rWVBQ2GnBmHfzi2NdFjAuk kerem@fedora
The key's randomart image is:
+---[RSA 4096]-----+
|  ..*=0 . |
|    *0.00 + |
|      0...+ . |
|      E.+ |
|  . S =.. |
| . = . .0 |
|o.. + .. o |
|*+=0.0. o |
|=0*=*=.. |
+---[SHA256]-----+
```

Рисунок 3.4: Создание RSA ключа

Затем был создан ключ ed25519 с помощью команды `ssh-keygen -t ed25519` (рис. Рисунок 3.5).


```
kerem@fedora:/var/tmp/kerem$ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/kerem/.ssh/id_ed25519):
/home/kerem/.ssh/id_ed25519 already exists.
Overwrite (y/n)? y
Enter passphrase for "/home/kerem/.ssh/id_ed25519" (empty for no passphrase)
Enter same passphrase again:
Your identification has been saved in /home/kerem/.ssh/id_ed25519
Your public key has been saved in /home/kerem/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:dts673IlnG+xvpbo6swNwa3H5ewklmc9NPY8H+1ptDA kerem@fedora
The key's randomart image is:
++--[ED25519 256]--+
|
|      .
|      S +....+ |
|    .  *+E=++ |
|      + B=XB* |
|     oo*o***+ |
|     .*0+=oo |
+----[SHA256]-----+
```

Рисунок 3.5: Создание ключа ed25519

3.3.2 3.2 Просмотр публичного ключа

Публичный ключ был выведен на экран командой `cat ~/.ssh/id_ed25519.pub` (рис. Рисунок 3.6).

```
kerem@fedora:/var/tmp/kerem$ gpg --full-generate-key
gpg (GnuPG) 2.4.8; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

Рисунок 3.6: Просмотр публичного ключа SSH

3.4 4. Создание PGP ключа (GPG)

Был создан GPG ключ командой `gpg --full-generate-key` (тип RSA, размер 4096, бессрочно) (рис. Рисунок 3.7).

```

Выберите тип ключа:
(1) RSA and RSA
(2) DSA and Elgamal
(3) DSA (sign only)
(4) RSA (sign only)
(9) ECC (sign and encrypt) *default*
(10) ECC (только для подписи)
(14) Existing key from card
Ваш выбор? 1
длина ключей RSA может быть от 1024 до 4096.
Какой размер ключа Вам необходим? (3072) 4096
Запрошенный размер ключа - 4096 бит
Выберите срок действия ключа.
    0 = не ограничен
    <n> = срок действия ключа - n дней
    <n>w = срок действия ключа - n недель
    <n>m = срок действия ключа - n месяцев
    <n>y = срок действия ключа - n лет
Срок действия ключа? (0)
Срок действия ключа не ограничен
Все верно? (y/N) y

GnuPG должен составить идентификатор пользователя для идентификации ключа.

Ваше полное имя: kerem
Адрес электронной почты: piriyeuwerkerem144@gmail.com
Примечание:
Вы выбрали следующий идентификатор пользователя:
    "kerem <piriyeuwerkerem144@gmail.com>"

Сменить (N)Имя, (C)Примечание, (E)Адрес; (O)Принять/(Q)Выход? o
Необходимо получить много случайных чисел. Желательно, чтобы Вы
в процессе генерации выполняли какие-то другие действия (печать
на клавиатуре, движения мыши, обращения к дискам); это даст генератору
случайных чисел больше возможностей получить достаточное количество энтропии.

```

Рисунок 3.7: Создание PGP ключа

3.4.1 4.1 Просмотр созданных ключей

Просмотр созданных ключей был выполнен с помощью команды `gpg --list-secret-keys --keyid-format LONG` (рис. Рисунок 3.8).

```

kerem@fedora:/var/tmp/kerem$ gpg --list-secret-keys --keyid-format LONG
gpg: проверка таблицы доверия
gpg: marginals needed: 3 completes needed: 1 trust model: pgp
gpg: глубина: 0 достоверных: 3 подписанных: 0 доверие: 0-, 0q, 0n, 0m, 0f, 3u
[keyboxd]
-----
sec  rsa4096/366909230651655D 2026-06-04 [SC]
      2CC66FBEA3629F16082EC3F3366909230651655D
uid          [ абсолютно ] kerem <piriyeuwerkerem144@gmail.com>
ssb  rsa4096/C383A6496BD20BCF 2026-06-04 [E]

sec  ed25519/D56AB2BC50B05D5A 2026-05-26 [SC]
      A0FAF8978C263375F5EF7A59D56AB2BC50B05D5A
uid          [ абсолютно ] kerem <1032254162@rudn.ru>
ssb  cv25519/3A8BD0233E8D0861 2026-05-26 [E]

sec  rsa4096/1EB978C2DFBA7060 2026-06-04 [SC]
      DFF934D7CE196744EBC0E5D81EB978C2DFBA7060
uid          [ абсолютно ] kerem <piriyeuwerkerem144@gmail.com>
ssb  rsa4096/61346D0CAFAAD1B6 2026-06-04 [E]

```

Рисунок 3.8: Просмотр GPG ключей

3.5 5. Добавление SSH ключа на GitHub

SSH ключ скопирован командой `cat ~/.ssh/id_ed25519.pub` и добавлен на GitHub в разделе Settings -> SSH and GPG keys -> New SSH key (рис. Рисунок 3.9).

```
kerem@fedora:~/work/study/2024-2025/Операционные системы$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZD11NTE5AAAAIPYmv73ESbLkXevoy2n0cR75p00EkspRMBHFXSqPxc kerem@fedora
kerem@fedora:~/work/study/2024-2025/Операционные системы$ ssh-keygen -t ed25519 -C "piriyewkerem144@gmail.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/kerem/.ssh/id_ed25519):
/home/kerem/.ssh/id_ed25519 already exists.
Overwrite (y/n)? y
Enter passphrase for "/home/kerem/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kerem/.ssh/id_ed25519
Your public key has been saved in /home/kerem/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:N0vJ8yblxRpF92xPQWlxxSA4Tovppnq0t9H5kcmUcbQ piriyewkerem144@gmail.com
The key's randomart image is:
+--[ED25519 256]--+
|      .o .++      |
|      +. o =|     |
|      =oE  +|     |
|      oooo o |    |
|      .S % . . |   |
|      oo% 0 . o.|  |
|      ..o* 0 o =|  |
|      . =. . + o.| |
|      .+o . . |   |
+-----[SHA256]-----+
```

Рисунок 3.9: Копирование и создание SSH ключа

3.6 6. Добавление GPG ключа на GitHub

GPG ключ был экспортирован командой `gpg --armor --export <XXXXXXXXXX>` и добавлен на GitHub. Успешность аутентификации проверена через `ssh -T git@github.com` (рис. Рисунок 3.10).

```
kerem@fedora:~/work/study/2024-2025/Операционные системы$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZD11NTE5AAAAIL3tr6LSFxUIhYnDdYkxZE9CXcgdVJvPMJsG0plzRcVo piriyewkerem144@gmail.com
kerem@fedora:~/work/study/2024-2025/Операционные системы$ ssh -T git@github.com
Hi piriyewkerem! You've successfully authenticated, but GitHub does not provide shell access.
```

Рисунок 3.10: Добавление GPG ключа и проверка ssh

3.7 7. Настройка подписи коммитов git

Были выполнены команды для настройки подписи коммитов: `git config --global user.signingkey <XXXXXXXXXX>`, `git config --global commit.gpgsign true` и `git config --global gpg.program $(which gpg2)` (рис. Рисунок 3.11).

```

kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$ git config --global commit.gpgsign true
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$ git config --global user.signingkey <отпечаток>
bash: синтаксическая ошибка рядом с неожиданным маркером «newline»
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$ git config --global commit.gpgsign true
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$ git config --global gpg.program $(which gpg2)
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$

```

Рисунок 3.11: Настройка подписи коммитов

3.8 8. Создание репозитория на GitHub

Был создан публичный репозиторий `study_2025-2026_os-intro` на платформе GitHub (рис. Рисунок 3.12).

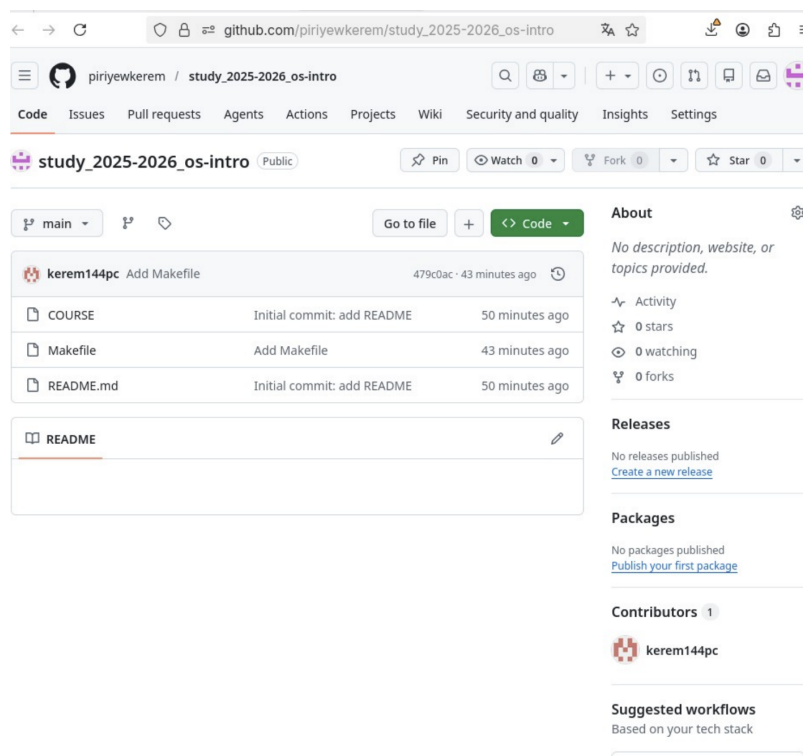


Рисунок 3.12: Созданный репозиторий на GitHub

3.9 9. Клонирование репозитория

Были созданы директории курса, и репозиторий был клонирован локально командой `git clone git@github.com:piriyewkerem/study_2025-`

2026_os-intro.git os-intro (рис. Рисунок 3.13).

```
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$ mkdir -p ~/work/study/2024-2025/"Операционные системы"
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$ cd ~/work/study/2024-2025/"Операционные системы"
kerem@fedora:~/work/study/2024-2025/Операционные системы$ git clone git@github.com:piriyekkerem/study_2025-2026_os-intro.git os-intro
fatal: целевой путь «os-intro» уже существует и не является пустым каталогом.
kerem@fedora:~/work/study/2024-2025/Операционные системы$
```

Рисунок 3.13: Клонирование репозитория

3.10 10. Настройка каталога курса

Был выполнен переход в каталог os-intro, создан файл COURSE, файлы добавлены в индекс (git add .), зафиксированы (git commit -m "Initial commit") и отправлены на сервер (git push) (рис. Рисунок 3.14).

```
kerem@fedora:~/work/study/2024-2025/Операционные системы$ cd os-intro
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$ echo os-intro > COURSE
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$ git add .
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$ git commit -m "Initial commit"
Текущая ветка: main
Эта ветка соответствует «origin/main».
нечего коммитить, нет изменений в рабочем каталоге
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$ git push
Everything up-to-date
kerem@fedora:~/work/study/2024-2025/Операционные системы/os-intro$
```

Рисунок 3.14: Отправка коммита

4 Вывод

В ходе выполнения лабораторной работы я: * Установил и настроил Git и GitHub CLI. * Сгенерировал SSH и GPG ключи. * Добавил ключи в аккаунт GitHub. * Настроил автоматическую подпись коммитов. * Создал удалённый репозиторий и клонировал его локально. * Настроил рабочее пространство для выполнения лабораторных работ.

Таким образом, цель работы достигнута: я освоил базовые навыки работы с системой контроля версий Git.

5 Контрольные вопросы

5.1 1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначены?

Системы контроля версий — это программные инструменты, которые отслеживают изменения в файлах, позволяют хранить историю изменений, возвращаться к предыдущим версиям, работать над проектом несколькими разработчиками одновременно, разрешать конфликты.

5.2 2. Объясните следующие понятия VCS и их отношения: хранилище, commit, история, рабочая копия.

- Хранилище (repository) — база данных, содержащая все версии файлов.
- Commit — фиксация изменений, создающая новую версию.
- История (history) — цепочка коммитов, показывающая эволюцию проекта.
- Рабочая копия (working copy) — текущая версия файлов, с которой работает пользователь.

5.3 3. Что представляют собой и чем отличаются централизованные и децентрализованные VCS?

Приведите примеры VCS каждого вида.

- Централизованные (CVS, Subversion): есть единый центральный сервер, все изменения отправляются на него.
- Децентрализованные (Git, Mercurial): каждый разработчик имеет полную копию репозитория, можно работать без подключения к серверу.

5.4 4. Опишите действия с VCS при единоличной работе с хранилищем.

Создание локального репозитория (`git init`), добавление файлов (`git add`), фиксация изменений (`git commit`), просмотр истории (`git log`), просмотр различий (`git diff`).

5.5 5. Опишите порядок работы с общим хранилищем VCS.

Клонирование удалённого репозитория (`git clone`), получение изменений (`git pull`), создание коммитов, отправка изменений (`git push`), разрешение конфликтов при необходимости.

5.6 6. Каковы основные задачи, решаемые инструментальным средством git?

- Отслеживание истории изменений.

- Работа с ветками.
- Слияние изменений.
- Совместная разработка.
- Откат к предыдущим версиям.

5.7 7. Назовите и дайте краткую характеристику командам `git`.

- `git init` — создание нового репозитория.
- `git add` — добавление файлов в индекс.
- `git commit` — фиксация изменений.
- `git push` — отправка изменений на удалённый сервер.
- `git pull` — получение изменений с сервера.
- `git clone` — копирование удалённого репозитория.
- `git branch` — работа с ветками.
- `git merge` — слияние веток.
- `git log` — просмотр истории.

5.8 8. Приведите примеры использования при работе с локальным и удалённым репозиториями.

Локальный: `git init`, `git add .`, `git commit -m "message"`.

Удалённый: `git clone git@github.com:user/repo.git`, `git pull`, `git push`.

5.9 9. Что такое и зачем могут быть нужны ветви (branches)?

Ветки — это независимые линии разработки. Нужны для: * Разработки новых функций без влияния на основной код. * Исправления ошибок. * Экспериментов. * Параллельной работы нескольких разработчиков.

5.10 10. Как и зачем можно игнорировать некоторые файлы при commit?

Создаётся файл `.gitignore`, в котором перечисляются шаблоны имён файлов/каталогов. Это нужно, чтобы не включать в репозиторий временные файлы, служебные файлы ОС, файлы с паролями, скомпилированные объектные файлы и т.д.

Список литературы

1. *Newham C.* Learning the bash Shell: Unix Shell Programming. — O'Reilly Media, 2005. — 354 p. — (In a Nutshell). — ISBN 0596009658. — URL: <http://www.amazon.com/Learning-bash-Shell-Programming-Nutshell/dp/0596009658>.
2. *Robbins A.* Bash Pocket Reference. — O'Reilly Media, 2016. — 156 p. — ISBN 978-1491941591.
3. *Zarrelli G.* Mastering Bash. — Packt Publishing, 2017. — 502 p. — ISBN 9781784396879.
4. *Таненбаум Э., Бос Х.* Современные операционные системы.. — 4-е изд. — СПб. : Питер, 2015.. — 1120 с. — (Классика Computer Science).